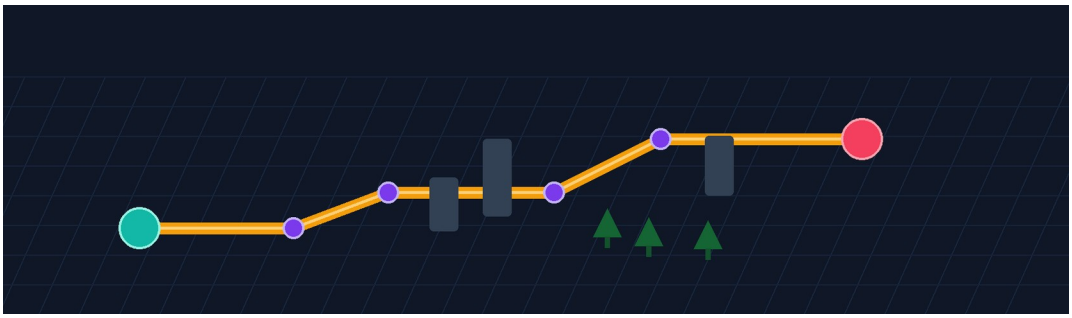
PROJECT REPORT

PATHFINDER

An Interactive 3D Pathfinding and Maze Visualization System



Submitted in partial fulfillment of the requirements for

DSA Bootcamp

College	Lovely Professional University
Programme	B.Tech CSE AI/ML
Faculty	Mr. Mahipal Singh Papola and Deepak Kumar
Submission Date	19 July 2026

Prepared by

Team Member	Registration Number
Lovejeet Singh	12521613
Sachit Babbar	12521201
Mohammad Asim	12518421
Shubham Kumar	12502181
Vineetosh Kumar	12502490

CONTENTS

Illustration Index

All tables are numbered and captioned in their corresponding sections.

1. ABSTRACT

Pathfinder is an interactive web application that teaches graph traversal, shortest-path search, weighted routing, and maze generation through a responsive 3D visual environment. The project transforms a rectangular grid into a four-directional graph where users can set a start and destination, place impassable walls, add cost-five weighted terrain, select an algorithm, and observe both the explored frontier and the returned route. Its central educational aim is to make algorithmic trade-offs visible: a route that is short in number of steps is not necessarily the least expensive route, and an algorithm that appears quick may sacrifice optimality.

The application implements Breadth-First Search, Depth-First Search, Dijkstra's algorithm, A* search, Greedy Best-First Search, and Bidirectional Breadth-First Search. It also implements Recursive Division, Recursive Backtracker, Randomized Prim's, Randomized Kruskal's with Union-Find, and Random Scatter maze generation. The main visualizer uses React Three Fiber and Three.js to render a 25 by 50 grid as instanced road tiles, buildings, weighted tree terrain, and endpoint beacons. The user interface separates calculation from playback: the algorithms return a visited-node sequence and a path, and an animation engine then presents current, visited, and final route states with different colors and speed controls.

Pathfinder is built with Next.js, React, TypeScript, Tailwind CSS, Zustand, Three.js, GSAP, Recharts, Shiki, Drizzle ORM, and optional Neon PostgreSQL persistence for shared grids. The report documents the implemented architecture, the data structures that support the algorithms, correctness conditions for weighted and unweighted routing, maze-generation behavior, functional validation, constraints, and future scope. Results are intentionally reported as functional and scenario-specific observations rather than fabricated device-independent benchmarks. The project was designed by a five-member team for the DSA Bootcamp course at Lovely Professional University.

Keywords: *pathfinding*, *maze generation*, *graph algorithms*, *3D visualization*, *A**, *Dijkstra*, *BFS*, *Union-Find*, *Next.js*.

2. INTRODUCTION

2.1 Background and Motivation

Pathfinding is a fundamental graph problem with direct applications in routing, robotics, games, navigation systems, network analysis, warehouse automation, and artificial intelligence. Although the theory behind graph search can be presented with pseudocode and asymptotic notation, students often find it difficult to connect those abstractions to the actual behavior of a search frontier. Pathfinder addresses that gap by showing how a search explores a space, where it spends work, and why the final route differs across algorithms.

The project models a board as a four-directional grid graph. Every traversable cell is a vertex; an admissible move to the cell above, right, below, or left is an edge; walls remove vertices from the traversable graph; and weighted tree terrain changes the cost of entering a cell. This deliberately constrained model is sufficient to explain the most important distinctions: unweighted shortest-hop search, weighted least-cost search, heuristic guidance, non-optimal exploration strategies, and the influence of graph topology.

Educational focus: The system is not intended to replace a GIS or production navigation engine. Its value lies in transparent, interactive explanation of data-structure behavior, correctness conditions, and visible trade-offs in graph algorithms.

Report note

2.2 Problem Statement

Traditional explanations of pathfinding usually show a final route but hide the intermediate search process. This can lead to misconceptions, such as assuming every algorithm always finds the same route, assuming the visually shortest route is automatically the cheapest, or assuming an algorithm that reaches the goal early is optimal. The project therefore needed a single environment in which a learner can configure the same board, run different algorithms, observe visitation order, inspect route metrics, and compare results without changing the problem definition.

The problem also includes maze generation. A useful visualizer must produce different obstacle topologies, animate their construction, preserve the endpoints, and communicate that structured maze generators can ensure a usable start-to-end route while random obstacle fields can deliberately create no-path conditions. The implementation must remain responsive on a default 25 by 50 board while supporting interaction, animation control, and optional sharing.

2.3 Objectives

1. Build an accessible web interface that represents a 25 by 50 four-directional grid as an interactive 3D environment.
2. Implement six pathfinding algorithms and correctly distinguish shortest-hop, least-cost, and non-optimal behavior.
3. Implement four structured maze generators plus random obstacle scattering and animate the maze-construction process.
4. Make terrain and visualization states visually distinct through a coherent color system, 3D geometry, legends, statistics, and controls.
5. Provide Learn and Compare pages to connect implementation behavior with pseudocode, complexity, and direct algorithm comparison.
6. Enable optional Share Grid persistence without making a database required for the core visualizer, Learn, or Compare experience.
7. Prepare a clean deployable Next.js application suitable for hosting on Vercel.

2.4 Scope and Boundaries

The current project focuses on deterministic grid-based learning experiences. Movement is limited to orthogonal directions, edge weights are represented as cell-entry costs, and the default board dimensions are fixed at 25 rows by 50 columns. The visualizer supports moving the endpoints using dedicated Start and End tools, but it does not expose arbitrary board resizing, diagonal movement, or real-world geographic routing data. The Compare page uses two synchronized 2D DOM mini-grids on cloned versions of the same board; it is not yet a dual-3D-scene comparison engine.

The Share Grid feature stores a bounded semantic representation: title, selected algorithm, wall positions, and weighted terrain. It does not persist completed animation states, camera position, arbitrary endpoint coordinates, or custom dimensions. This conservative scope reduces payload size and ensures that shared boards can be restored safely to the default grid. The optional database branch fails gracefully when a deployment has no DATABASE_URL configuration.

2.5 Methodology

The development approach combined algorithm research, component-based UI engineering, controlled visual-state design, source-level verification, and deployment preparation. First, each pathfinding and maze algorithm was defined against the common grid model. Next, reusable data structures such as a binary MinHeap and Union-Find were implemented where their algorithmic guarantees were required. The React interface and three-dimensional scene were then connected to a centralized Zustand store, so that editing, calculation, animation, statistics, and reset behavior followed a consistent state model. Finally, the project was checked with TypeScript, ESLint, production build, database-configuration, and

dependency-audit commands, while behavioral cases were used to validate the conditions under which each algorithm is optimal.

Algorithm behavior was evaluated through controlled board scenarios rather than through a single favorable demonstration. A uniform-cost board with a clear route was used to establish minimum-hop behavior; obstacle layouts with dead ends and alternate corridors were used to observe breadth of exploration; and weighted-tree placements were used to distinguish a visually shorter route from a lower-cost route. The same start position, end position, walls, terrain costs, and movement settings were held constant whenever algorithms were compared, so differences in visited nodes, path length, path cost, and computation time could be attributed to the selected search strategy.

Each scenario was checked in both its visual and numerical forms. The board had to preserve readable distinctions among the current node, visited cells, final route, walls, weighted trees, and start/end markers during animation. The result panel was then reviewed to confirm that the reported metric matched the run: unweighted searches were judged by route reachability and minimum-hop expectations, while weighted searches were judged by the accumulated terrain cost as well as the reconstructed path. Maze generation was reviewed separately for endpoint preservation, wall topology, and the ability to run a pathfinding algorithm on the generated board afterward.

The final review also covered normal user actions: changing the selected algorithm, editing the grid, clearing transient path states, resetting the board, adjusting playback speed, opening Learn and Compare views, and loading the deployed experience in a browser. Release commands and production builds were checked as manual evidence for a submission-ready application. This methodology intentionally separates those checks from automated regression testing or continuous integration, neither of which is represented as a committed feature of the current repository.

3. SYSTEM DESIGN AND IMPLEMENTATION

3.1 Architectural Overview

Pathfinder is a single Next.js App Router application. It is not a monorepo: the front-end pages, algorithm modules, rendering components, optional route handlers, database schema, and deployment configuration are organized in one TypeScript codebase. The application has four primary pages: a landing page, the main 3D visualizer, a Learn page, and a Compare page. The core visualization route dynamically imports the Three.js scene with server-side rendering disabled, because WebGL and browser interaction APIs are client-side concerns.

The store is the central coordinator. It owns the grid, endpoint coordinates, selected algorithm, selected maze generator, speed, active tool, run flags, statistics, and references to animation controllers. The architectural separation is deliberate: algorithm modules calculate outcomes; animation modules sequence visual states; React components render controls and information; and the 3D scene maps grid state into visible meshes. This structure prevents visual timing concerns from changing the correctness logic of the algorithms.

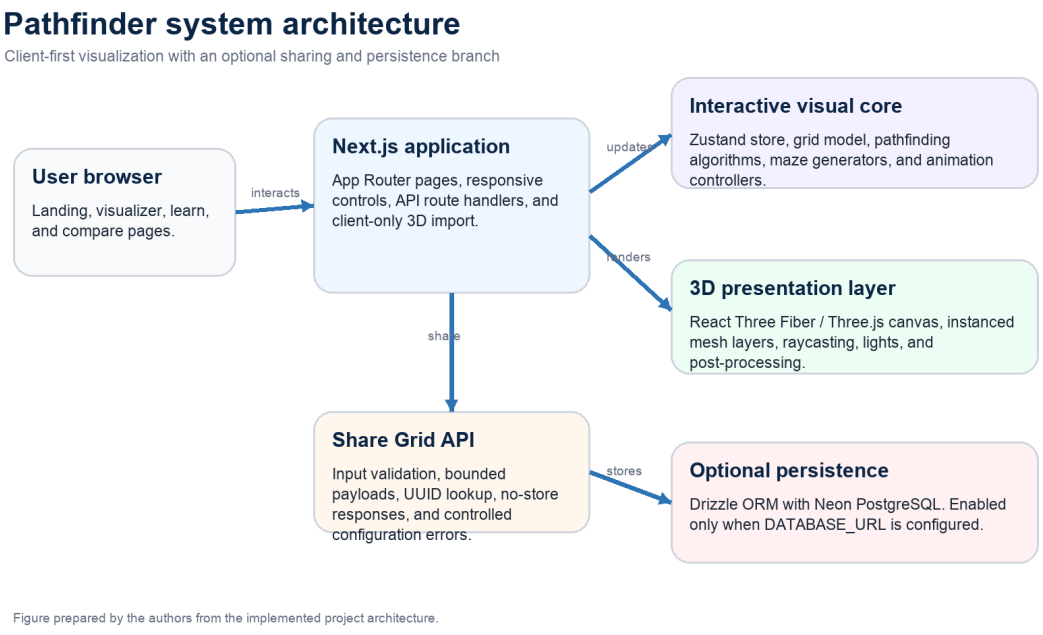


Figure 1. Pathfinder system architecture.

Source: Authors, based on the implemented project architecture.

3.2 Technology Stack

Table 1. Technology stack and use in the project.

Layer	Technology	Role in Pathfinder
Application framework	Next.js 16.2.10, React 19, TypeScript	App Router routes, component model, type-safe implementation, and production build.
Styling and UI	Tailwind CSS 4, Lucide React, Framer Motion	Responsive glass-style controls, icons, navigation, and reduced-motion-aware transitions.
3D rendering	Three.js, React Three Fiber, Drei, Postprocessing	Client-side WebGL canvas, instanced mesh layers, camera controls, lighting, bloom, and vignette.
State and animation	Zustand, GSAP	Central visualizer/compare state and timeline-driven search or maze playback.
Learning and compare	Shiki, Recharts	Syntax-highlighted pseudocode, step accordions, complexity content, and comparison charts.
Persistence	Drizzle ORM, Neon serverless PostgreSQL	Optional Share Grid schema, validation-backed API persistence, and retrieval.
Deployment	Vercel	Hosting of the Next.js application and optional environment-variable configuration.

Source: Authors; package versions reflect the audited implementation.

3.3 Grid Model and Visual Semantics

The default grid contains 1,250 cells (25 rows multiplied by 50 columns). With four-directional movement, a cell may connect only to its orthogonal neighbors. In an empty default board, the start cell at (12, 10) and the end cell at (12, 40) are 30 moves apart, so a direct route contains 31 displayed nodes. This distinction matters because the interface reports route length as the number of nodes in the returned path, whereas the number of moves is one less than that value.

Table 2. Grid model and state semantics.

Semantic state	Representation	Meaning / behavior
Road	Light slate tile (#CBD5E1)	Traversable terrain with entry cost 1.
Wall	Slate building (#334155)	Impassable obstacle; excluded from pathfinding neighbors.
Start / End	Teal / rose beacons (#14B8A6 / #F43F5E)	Origin and goal; protected during edits and maze generation.
Weighted terrain	Dark green tree	Traversable terrain whose entry cost is 5.
Current	Cobalt (#2563EB)	A cell actively highlighted during playback.
Visited	Royal violet (#7C3AED)	A cell explored by the selected algorithm.
Route	Amber (#F59E0B)	The final returned path after successful search.

Source: Authors; colors are from the implemented visualizer palette.

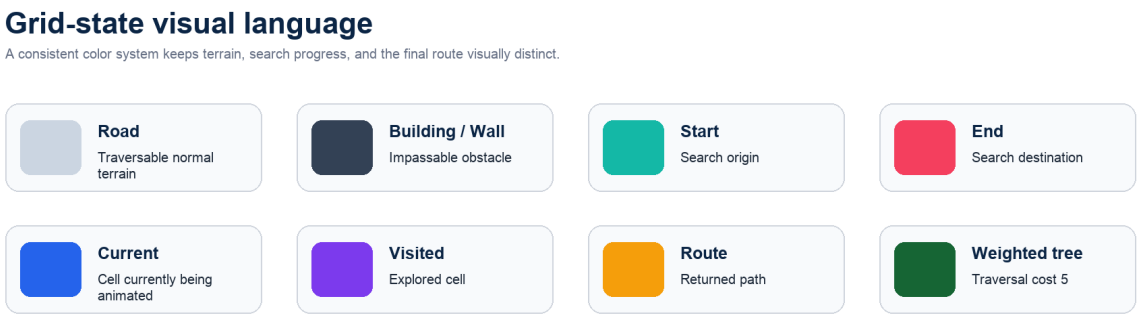


Figure 3. Grid-state visual language used by the Pathfinder visualizer.

Source: Authors.

3.4 Interaction, Calculation, and Animation Workflow

An invisible raycast plane converts a user interaction in the 3D scene into a grid cell. The active tool then creates or removes a wall, creates cost-five weighted terrain, erases a non-endpoint cell, or relocates an endpoint. Board editing is disabled during a running visualization so that an algorithm operates on a stable graph. On Visualize, the store first cancels prior playback and clears transient search fields while preserving the semantic board configuration.

The selected algorithm runs synchronously and returns a visitation sequence, a reconstructed route, and a found flag. The application measures its compute-only duration using performance.now(); that number excludes animation. The AnimationEngine subsequently applies typed-buffer state to cells: a current-cell flash, a settled visited state, and then a final route reveal. A monotonically increasing run identifier and engine identity checks prevent stale asynchronous callbacks from changing the state after Stop, Reset, or a new run. Pause, resume, stop, clear-path, reset, and speed updates are playback controls rather than changes to the graph algorithm itself.

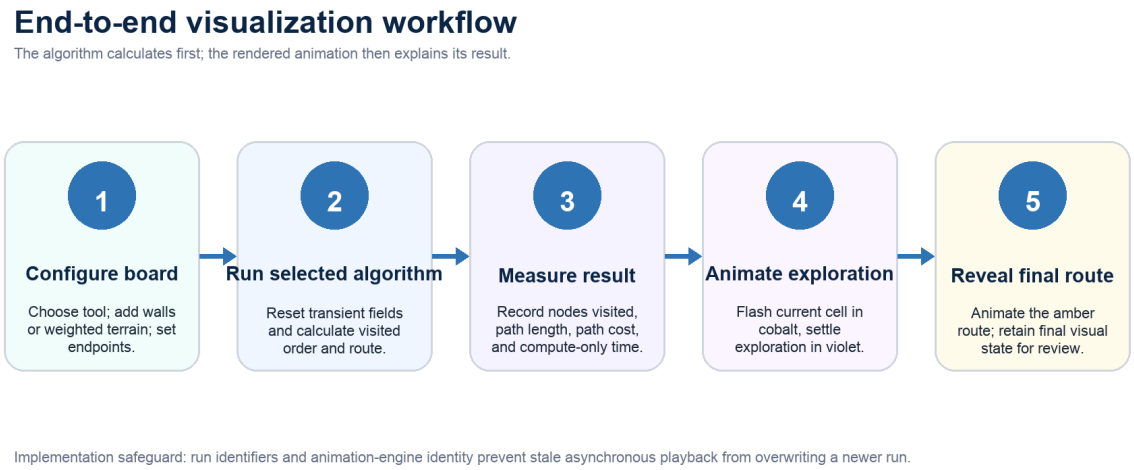


Figure 2. End-to-end visualization workflow.

Source: Authors.

3.5 Three-Dimensional Rendering Strategy

Rendering every grid cell as a separate React component would be unnecessarily expensive. The main scene therefore uses three instanced mesh layers: road tiles, building meshes for walls, and cone-like tree meshes for weighted terrain. Instancing permits a default board of 1,250 cells to be represented efficiently while allowing different scales, heights, and emissive states. Buildings receive deterministic height variation; endpoint beacons add person-like markers, glow discs, and floating indicators.

The Canvas uses controlled device pixel ratio, shadows, ACES filmic tone mapping, fog, OrbitControls, directional lighting, ambient illumination, bloom, and vignette. These presentational details support legibility without changing the underlying algorithms. The color palette is intentionally distinct: cobalt marks the active node, violet marks prior exploration, and amber identifies the final route. This makes the three states distinguishable at first glance and avoids relying only on shape or text.

3.6 Learn, Compare, Share, and Deployment

The Learn page exposes six pathfinding and four maze cards. Each card includes an explanation, Shiki-highlighted pseudocode, a copy control, complexity information, and a Framer Motion accordion that narrates the main steps. The Compare page initializes one common board and runs two selected algorithms on independent deep copies, making the input fair while keeping their exploration states separate. It currently renders two 2D DOM mini-grids, a Recharts grouped bar chart for nodes visited, path length, and path cost, result cards, and generated comparison text. It should be described accurately as a side-by-side 2D comparison, not as a dual 3D scene.

Share Grid is an optional server feature. A POST request validates content type, JSON shape, a 100 KiB payload limit, title length, supported algorithm, grid-cell bounds, allowed terrain types, and positive weights. A GET request validates the UUID and revalidates stored data before it is restored. When DATABASE_URL is absent, the visualizer, Learn page, and Compare page remain usable, while sharing returns a controlled configuration error. The Vercel deployment uses secure response headers, hides the X-Powered-By header, and does not expose database secrets to the browser. The live deployment is available at https://pathfinding-visualizer-flax-one.vercel.app/.

Deployment boundary: Anonymous UUID-based sharing is a convenience feature rather than an authorization system. If public usage expands, authentication, ownership, rate limiting, monitoring, and expiry controls should be introduced.

Report note

4. ALGORITHM DESIGN AND ANALYSIS

4.1 Graph Formulation and Evaluation Criteria

Let R be the number of rows and C the number of columns. The grid contains V = R x C cells. Each traversable cell can have at most four directed neighbor relationships, so E is bounded by 4V and the common O(V + E) traversals are O(RC) on this application grid. A normal cell has entry cost 1 and a weighted tree cell has entry cost 5. The path cost is the sum of all entered cell weights after the start cell; the start node itself is not charged.

The report uses four separate performance concepts. Completeness asks whether the algorithm can find a route when a reachable route exists. Minimum-hop optimality asks whether it returns the fewest moves on a uniform-cost board. Least-cost optimality asks whether it minimizes the sum of terrain costs on the supported non-negative model. Exploration efficiency refers to nodes visited or measured computation on the same board; it is useful for comparison, but it is not a universal substitute for optimality. [1]

Measurement rule: The interface shows path length as the count of nodes in the returned route. If a route contains L nodes, its number of moves is L - 1. The timing metric measures algorithm computation only, while the separately visible playback duration depends on animation speed.

Report note

4.2 Breadth-First Search (BFS)

Breadth-First Search explores a graph in layers of increasing hop distance from the start. Pathfinder implements BFS using an array-backed FIFO queue with a moving head index, avoiding costly array shifting. A node is marked visited when it is enqueued, and a previous-node pointer is stored so that the final path can be reconstructed after the end is reached.

On an unweighted or uniform-cost grid, the first discovered route to the end uses the fewest moves because BFS completely explores distance d before distance d + 1. Its time complexity is O(V + E) and its space complexity is O(V). However, the implementation deliberately ignores terrain weights. Therefore, a route with fewer steps can still be more expensive than an available detour; BFS should be selected when minimum hop count, not minimum terrain cost, is the requirement. [1]

4.3 Depth-First Search (DFS)

Depth-First Search uses a LIFO stack to follow one branch deeply before backtracking. In Pathfinder, neighbors are pushed in reverse order so that the effective traversal preference is up, right, down, then

left. The implementation marks nodes when popped and safely skips duplicate stack entries. Like BFS, DFS has O(V + E) time and O(V) space complexity on a graph.

DFS is useful for demonstrating deep exploration and reachability, but it does not compare candidate route length or cost. It terminates when it first reaches the goal, which can be through a long detour. It also ignores weighted terrain. It may appear fast in a favorable layout, but that outcome depends on obstacle geometry and neighbor ordering rather than a shortest-path guarantee. [1]

4.4 Dijkstra's Algorithm

Dijkstra's algorithm generalizes unweighted shortest-path search to non-negative costs. Pathfinder uses a custom binary MinHeap to store immutable queue entries of the form (node, distance). Whenever relaxation finds a better cost, a new entry is pushed; a later stale entry is discarded by comparing its stored distance with the node's current best distance. This avoids mutating a heap item in place and preserves simple priority-queue correctness.

Moving from a cell to its neighbor adds the neighbor's weight to the accumulated distance. Since all supported costs are positive and non-negative, the first non-stale extraction of a node from the minimum-priority queue finalizes the least known cost to that node. Hence Dijkstra returns a minimum-total-cost route. Its binary-heap implementation has O((V + E) log V) time and O(V) space complexity. On an all-unit-cost board it also returns a minimum-hop path, but BFS is generally simpler because it does not require heap operations. [1], [2]

4.5 A* Search

A* combines the actual cost accumulated so far with an estimate of remaining cost: f(n) = g(n) + h(n). Pathfinder uses Manhattan distance for h because movement is restricted to four orthogonal directions. The implementation multiplies that distance by the minimum traversable cell weight, making the heuristic safe even if a restored board contains weights smaller than one. Like Dijkstra, it uses immutable MinHeap entries and stale-entry checks.

For the supported grid model, the scaled Manhattan estimate does not overestimate the remaining cost, so it is admissible and consistent. A* therefore retains Dijkstra's least-cost guarantee for non-negative terrain while often exploring fewer irrelevant cells in goal-directed layouts. Its worst-case complexity remains O((V + E) log V), and it is not correct to claim it is always faster than Dijkstra: maze bottlenecks or a weakly informative heuristic can make their behavior similar. [1], [3]

4.6 Greedy Best-First Search

Greedy Best-First Search uses a MinHeap ordered only by the Manhattan heuristic h(n). It is strongly directed toward the end and uses a discovered set to avoid repeatedly scheduling the same cell. Because it omits g(n), the cost already paid, it does not revise a node when a better parent or less expensive route appears.

The algorithm can explore relatively few cells in open layouts, which makes it an effective educational contrast with A*. Its efficiency is heuristic and topology-dependent. It is neither minimum-hop optimal nor least-cost optimal: an apparently close path may be blocked by walls, and a direct route may cross expensive terrain. Its time and space complexities are O((V + E) log V) and O(V), respectively, with the project's binary heap.

4.7 Bidirectional Breadth-First Search

Bidirectional BFS runs one breadth-first search from the start and another from the end. Pathfinder maintains separate queues, visited sets, and parent maps, expands complete frontier levels alternately, and merges the two parent chains when the search regions meet. The grid is an undirected, symmetric movement model, so backward search from the end is valid.

On an unweighted grid, this technique remains minimum-hop optimal and has O(V + E) time and O(V) space complexity. In practice, two expanding fronts often meet before a one-sided BFS would reach the destination, so the visited region can be smaller. The exact saving is not fixed: corridors, obstacles, and the shape of frontiers determine the benefit. Like ordinary BFS, this implementation ignores weighted terrain and should not be described as least-cost optimal on a weighted board.

Table 3. Pathfinding algorithm property matrix.

Algorithm	Primary structure	Min-hop optimal?	Least-cost optimal?	Best use
BFS	FIFO queue	Yes, uniform cost	No	Reliable unweighted shortest route
DFS	LIFO stack	No	No	Deep exploration demonstration
Dijkstra	Binary MinHeap	Yes, equal costs	Yes, non-negative costs	Weighted shortest route
A*	Binary MinHeap	Yes, equal costs	Yes, non-negative costs	Goal-directed weighted route
Greedy	Binary MinHeap	No	No	Speed-versus-quality demonstration
Bidirectional BFS	Two FIFO queues	Yes, uniform cost	No	Unweighted two-front comparison

Source: Authors, based on the implemented algorithms and their correctness conditions.

Table 4. Pathfinding time and space complexity.

Algorithm	Time complexity	Space complexity	Why it can appear faster or slower
BFS	O(V + E)	O(V)	Layered expansion can visit a wide region before the goal.
DFS	O(V + E)	O(V)	A lucky deep branch can reach the goal early; dead ends can cause long detours.
Dijkstra	O((V + E) log V)	O(V)	Heap overhead buys correct cost-aware exploration.
A*	O((V + E) log V) worst case	O(V)	Heuristic often focuses work toward the goal, not on every layout.
Greedy	O((V + E) log V)	O(V)	Heuristic-only priority is directional but may get trapped by obstacles.
Bidirectional BFS	O(V + E)	O(V)	Two frontiers may meet early; benefit depends on topology.

Source: Complexity notation follows the graph model in Section 4.1.

Figure 9. Algorithm selection guide for the implemented grid model.

Source: Authors.

4.8 Supporting Data Structures

Binary MinHeap

The reusable MinHeap supports push and pop in O(log n) time and peek in O(1) time through sift-up and sift-down operations. It accepts a custom comparison function, allowing Dijkstra to prioritize accumulated distance, A* to prioritize g + h, Greedy search to prioritize h, and maze repair to prioritize the number of walls opened. The same abstraction demonstrates how a data structure can support several algorithms without coupling their correctness logic.

Union-Find / Disjoint Set

Randomized Kruskal's maze uses Union-Find to record which logical room cells are already connected. Path compression shortens future find operations, and union by rank prevents tall trees. Its amortized operations are near constant, conventionally written O(alpha(V)), where alpha is the very slowly growing inverse Ackermann function. The structure allows an inter-room wall to be removed only when it connects different components, preventing cycles in the logical maze graph. [6]

4.9 Maze Generation Design

Maze generation in Pathfinder produces an ordered sequence of wall or passage steps rather than rendering a final board instantaneously. The AnimationController applies the returned steps with speed-dependent delay, supports pause, resume, and cancellation, and protects start and end cells from being overwritten. This makes maze algorithms observable as construction processes, not only as static obstacle patterns.

Recursive Division

Recursive Division begins with open space, adds outer borders, places a horizontal or vertical wall based on chamber dimensions, leaves one opening in each dividing wall, and recurses into the resulting subchambers. It produces architectural rooms, long straight boundaries, and visible corridors. The core construction is O(RC) in time and space for the finite grid and recorded step list.

Recursive Backtracker

Recursive Backtracker begins with walls, chooses an odd-coordinate logical room, then uses a depth-first stack to carve two-cell jumps and the wall between them. When no unvisited neighbor remains, it backtracks. The characteristic result is a long, winding, depth-first corridor structure with fewer short branches. Its core construction is O(RC) time and O(RC) space.

Randomized Prim's

Randomized Prim's begins with a wall-filled board and grows a connected passage region from a random odd-coordinate seed. It maintains a frontier of unvisited logical room cells, randomly selects a frontier cell, connects it to an existing passage neighbor, and expands the frontier. The result usually has branching, tree-like structure and many short dead ends. The core construction is O(RC) time and O(RC) space in this grid implementation. [4]

Randomized Kruskal's with Union-Find

Randomized Kruskal's treats odd-coordinate rooms as logical graph vertices. It constructs candidate walls between neighboring rooms, shuffles them with Fisher-Yates, and removes a wall only when Union-Find reports that the rooms are in distinct components. The resulting logical graph is a randomized spanning tree before endpoint repair. Its time is O(RC alpha(RC)), commonly treated as approximately O(RC), and it uses O(RC) space. [5], [6]

Random Scatter

Random Scatter independently turns non-endpoint cells into walls at a default density of 0.30, clamped between 0.10 and 0.50. It is an O(RC) random obstacle generator rather than a structured maze. Unlike the four structured generators, it intentionally does not apply reachability repair. Therefore it can create a valid no-path test case and can trigger the visualizer's no-path feedback.

Table 5. Maze generator characteristics.

Generator	Construction idea	Core complexity	Visual character	Endpoint behavior
Recursive Division	Split open chambers with walls and a passage	O(RC)	Rooms and long walls	Repaired if needed
Recursive Backtracker	Depth-first carving through a wall field	O(RC)	Long winding corridors	Repaired if needed
Randomized Prim	Random frontier growth from a seed	O(RC)	Branching tree / dead ends	Repaired if needed
Randomized Kruskal	Connect disjoint rooms with Union-Find	O(RC alpha(RC))	Distributed spanning-tree pattern	Repaired if needed
Random Scatter	Independent random wall placement	O(RC)	Irregular obstacle field	No reachability guarantee

Source: Authors; "repaired" refers to ensureReachableMaze described in Section 4.10.

4.10 Reachability Repair and Correctness Nuance

The four structured generators call ensureReachableMaze after producing their primary steps. The helper first applies the proposed maze conceptually to a passability map, forces the endpoints open, and uses BFS to check whether a route exists. If not, it performs a minimum-wall-opening search: entering

an existing passage has cost 0 and entering a wall has cost 1. The MinHeap then finds a repair route that opens the fewest required walls, and passage steps are appended to the animation.

This design guarantees that Recursive Division, Recursive Backtracker, Randomized Prim's, and Randomized Kruskal's produce a usable start-to-end board for the current endpoints. The repair step can take O(RC log(RC)) in the disconnected case. It must not be used to claim that the final displayed board always remains a mathematically perfect maze with exactly one route between every pair of cells: endpoint reachability is intentionally prioritized over that formal invariant. Random Scatter remains unrepaired by design.

Correctness summary: BFS and Bidirectional BFS are optimal for shortest-hop paths only when all traversable cells have equal cost. Dijkstra and A* are optimal for the project's non-negative weighted model. DFS and Greedy Best-First Search may return valid paths, but do not guarantee a minimum-hop or minimum-cost result.

Report note

5. RESULTS AND DISCUSSION

5.1 Evaluation Approach

The project is primarily an interactive teaching application, so results are reported as functional outcomes on controlled scenarios rather than as universal hardware-independent benchmark claims. Each pathfinding execution reports nodes visited, route length, path cost, success status, and compute-only time. The last metric is captured with performance.now() before the GSAP playback begins. It is useful for a same-browser comparison on the same grid, but it varies with device, browser, grid layout, and runtime conditions; it should not be interpreted as a laboratory benchmark.

Validation focused on algorithm conditions that matter to users. An unweighted test checks whether shortest-hop algorithms agree; a weighted case checks whether cost-aware algorithms avoid an expensive direct route; structured maze samples check endpoint reachability after repair; and random scatter demonstrates that no-path states are possible and are surfaced intentionally. TypeScript strict compilation, ESLint, database configuration checks, production dependency audit, and the production build were also used to validate deployment readiness. The repository deliberately does not include a regression-test suite or CI workflow, so this report does not claim automated test coverage.

Table 6. Functional validation scenarios.

Scenario	Expected behavior	Observed functional result	Interpretation
500 random unweighted 9 x 11 boards	BFS and Bidirectional BFS should agree on route existence and minimum-hop length.	The bidirectional implementation matched BFS for route existence and returned shortest-hop length across the sampled layouts.	Supports the implementation's uniform-cost optimality behavior.
Weighted constructed case	A lower-cost detour should be preferred by Dijkstra and A* even if it has more hops.	For a 4-hop cost-16 direct route and a 6-hop cost-6 detour, Dijkstra and A* selected the lower-cost detour; BFS, Bidirectional BFS, and Greedy selected the short-hop expensive route.	Shows the difference between hop count and path cost.
100 generated samples per structured maze	Endpoints should remain connected after reachability repair.	Recursive Division, Backtracker, Prim, and Kruskal each produced a reachable route in the sampled runs.	Confirms the intended effect of ensureReachableMaze.
Random Scatter	A route is not guaranteed.	Some generated boards can disconnect the endpoints and trigger the no-path state.	This is intentional for stress testing and teaching failure cases.

Source: Authors; results describe targeted behavior checks, not device-independent performance benchmarks.

5.2 Interpretation of Algorithm Behavior

On an empty default board, the direct Manhattan separation from (12, 10) to (12, 40) is 30 moves. A correctly returned direct path therefore contains 31 nodes in the user interface. In an unweighted environment, BFS creates an expanding wave and guarantees the fewest moves, while Bidirectional BFS commonly reduces the visible explored area by allowing two waves to meet. DFS can visually commit to a long corridor, and Greedy Best-First Search can head directly toward the destination but be misled by barriers. These outcomes are not defects; they make the trade-off between completeness, optimality, and goal direction explicit.

Weighted terrain produces the most important contrast. A direct route that crosses weighted trees may have fewer hops but a higher total cost. BFS, DFS, Greedy, and Bidirectional BFS do not use cost as their decision criterion in this project, even though the interface can calculate the cost of the route they return. Dijkstra and A* do use accumulated cost, and A* adds an admissible heuristic to guide its exploration. The result is a clear explanation of why the “shortest” route must be defined carefully: shortest in moves and cheapest in cost are different objectives.

How to read the results panel: Nodes visited indicates search exploration, Path Length counts returned route nodes, Path Cost sums entered terrain weights, and Time is compute-only time. A lower visited count is a useful visualization metric, but it is not by itself a universal definition of a better algorithm.

Report note

5.3 User-Interface Evidence

The following placeholders are intentionally included for the team to replace with screenshots from the deployed application before submission. They are placed near the results discussion so that visual evidence directly supports the descriptions in this report. Each screenshot should be exported at clear resolution and should retain the legend or relevant metrics whenever possible. Captions may be adjusted to match the exact final screenshots.

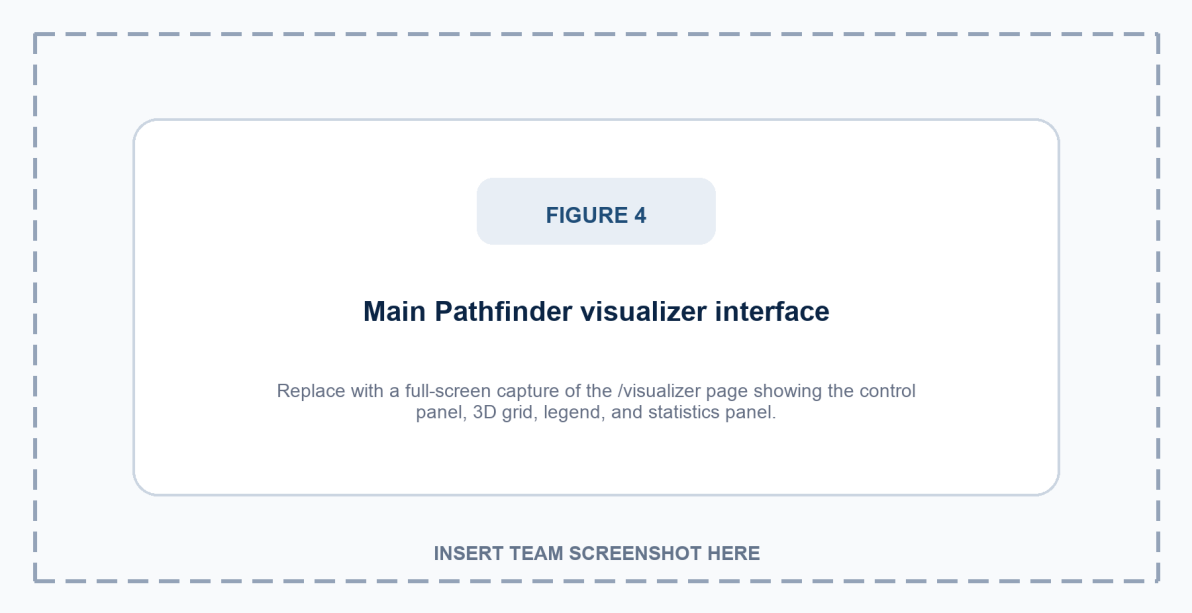


Figure 4. Screenshot placeholder: main Pathfinder visualizer interface.

Source: To be replaced by a team-captured project screenshot.

Recommended capture: open the visualizer with the control panel visible, a readable 3D grid, the legend, and statistics. This image demonstrates the integration of interaction tools, rendering, and feedback in one view.

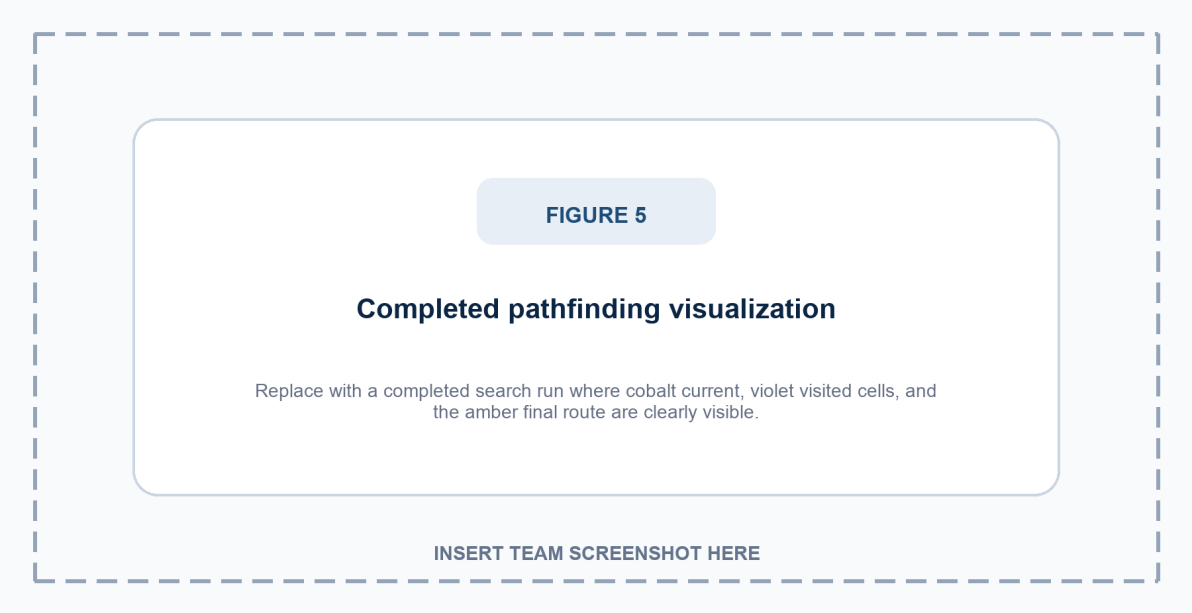


Figure 5. Screenshot placeholder: completed pathfinding visualization.

Source: To be replaced by a team-captured project screenshot.

Recommended capture: run an algorithm on a nontrivial grid where violet visited cells and the amber final route remain clearly distinguishable. Include the result card or legend if possible so the screenshot verifies the visual-state mapping described in Sections 3 and 5.

5.4 Maze, Learn, and Compare Evidence

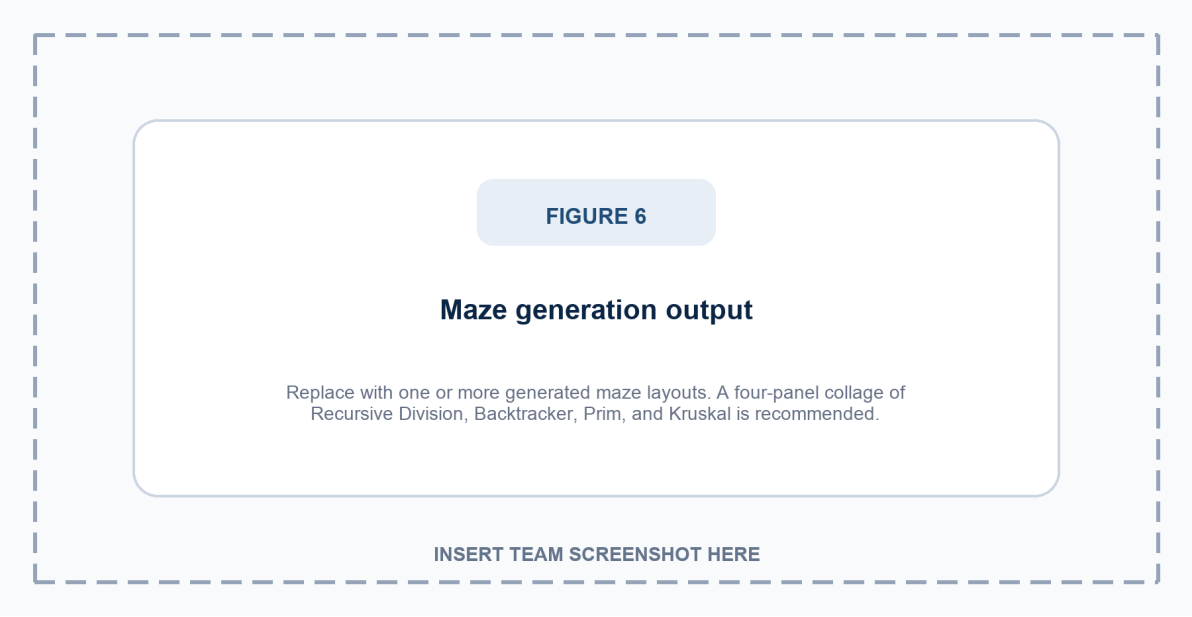


Figure 6. Screenshot placeholder: maze generation output.

Source: To be replaced by a team-captured project screenshot.

Recommended capture: take four separate maze screenshots and create a simple 2 by 2 collage, or insert one representative maze with the generator control visible. Label the panels Recursive Division, Recursive Backtracker, Randomized Prim's, and Randomized Kruskal's. This will make the differing visual topology of the generators easy for the examiner to compare.

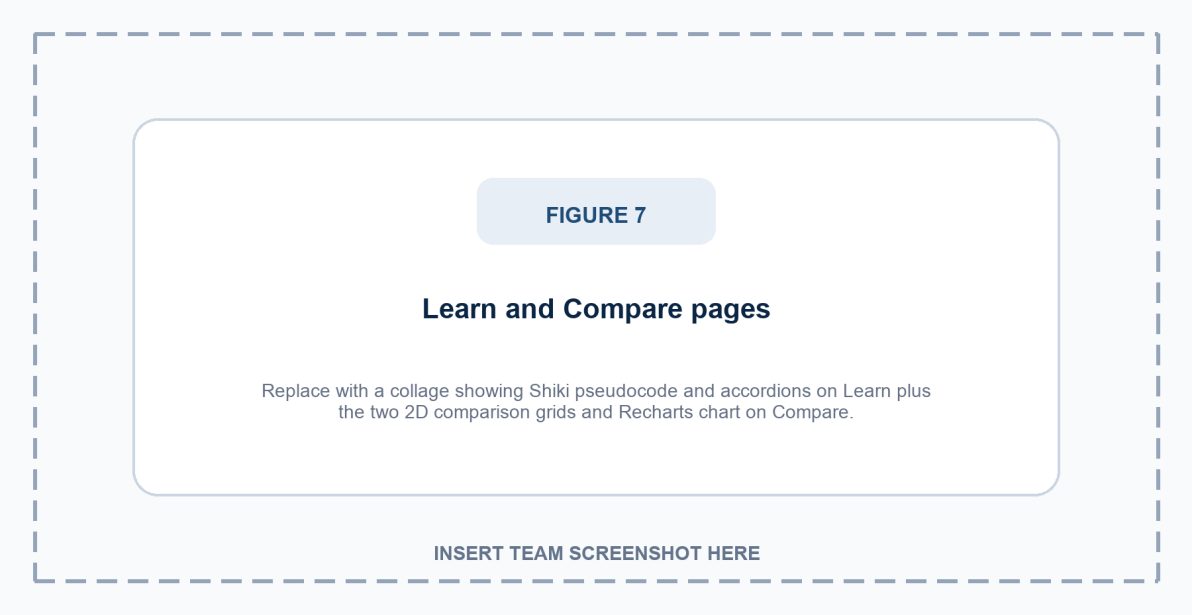


Figure 7. Screenshot placeholder: Learn and Compare pages.

Source: To be replaced by a team-captured project screenshot.

Recommended capture: use a collage. For Learn, show an AlgorithmCard with Shiki-highlighted pseudocode and a step-by-step accordion. For Compare, show its two 2D mini-grids, the same shared input board, the Recharts bar chart, and generated insight. Do not describe this current feature as dual 3D visualization.

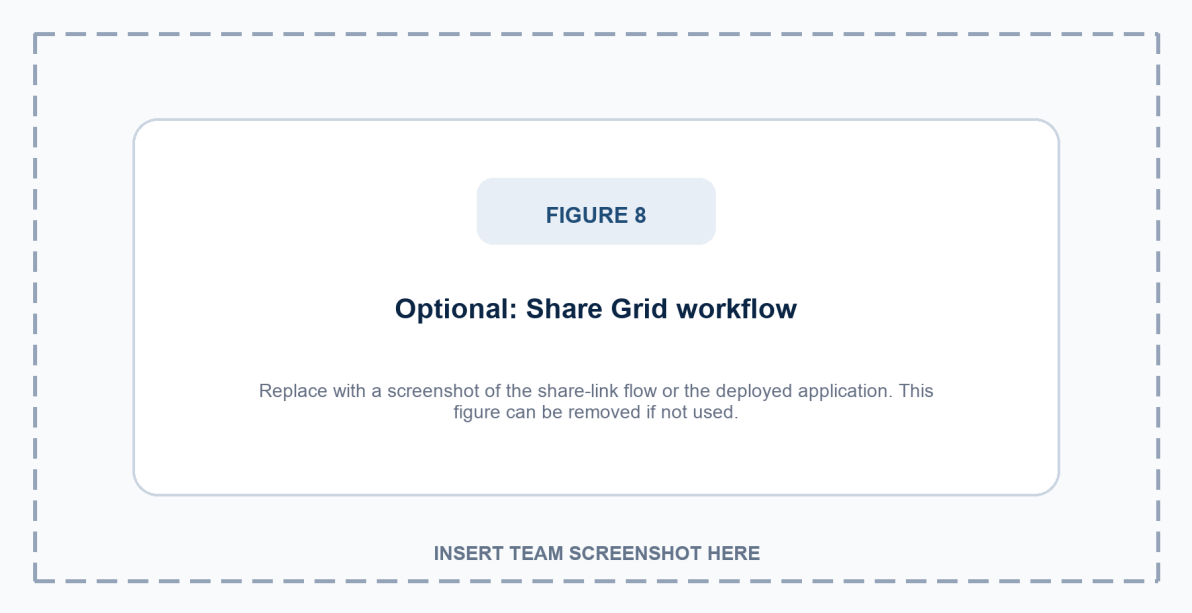


Figure 8. Optional screenshot placeholder: Share Grid workflow.

Source: Optional team-captured project screenshot.

Recommended capture: show a generated share link or the live Vercel deployment. This figure is optional; remove it if the project is deployed without DATABASE_URL and Share Grid is not being demonstrated.

5.5 Discussion of Strengths and Constraints

The project's main strength is the connection between correct algorithmic logic and legible visual feedback. The shared grid model means that differences across algorithms arise from their actual strategies rather than from different data. The 3D main visualizer provides spatial clarity, while the Learn page connects code-like explanations to complexity and steps. Optional sharing is kept separate from the core interaction path, so the visualizer is functional even without a configured database.

Several constraints are intentional or remain future work. Compare currently executes algorithms sequentially on cloned grids and visualizes them in 2D DOM panels, not in synchronized 3D scenes. Compare does not yet expose weighted terrain or endpoint editing. Browser timing is pedagogical rather than benchmark-grade. Share links are anonymous, have no ownership model, and only preserve the default-size semantic board. Finally, a WebGL-dependent interface benefits from a future lightweight fallback for low-power or no-WebGL environments.

6. USER WORKFLOW, DEPLOYMENT, AND PROJECT ACCESS

6.1 End-to-End User Workflow

Pathfinder is designed around a repeatable learning loop. A learner selects a search method, constructs a scenario by placing walls or weighted trees, adjusts the endpoints when required, and starts either a pathfinding algorithm or a maze generator. Before playback begins, the selected algorithm computes a visitation order, a predecessor-based route, and result metadata. The animation controller then applies those state changes so the learner can distinguish cells evaluated during search from the cells that form the final route.

The workflow is deliberately reversible. Clear Path preserves the board while removing transient current, visited, and route states; Reset restores a fresh default grid; and maze generation establishes a new obstacle topology while protecting the endpoints. This separation enables a student to alter one condition at a time, rerun the same algorithm, and observe how walls, weighted terrain, and movement constraints influence the search strategy rather than merely viewing a static answer.

6.2 Fair Comparison and Result Interpretation

A meaningful comparison begins with the same grid, start and end locations, terrain costs, and movement rules. The Compare page runs algorithms on independent clones of a shared input board, so one traversal cannot mutate the data used by the other. Its result panels are intentionally implemented as two-dimensional DOM mini-grids rather than synchronized 3D scenes; this preserves fairness of the input data while accurately describing the current feature boundary.

The result metrics are complementary rather than a single quality score. Nodes Visited measures the extent of search exploration; Path Length counts nodes in the reconstructed route; Path Cost sums the entered terrain weights; and Time is the compute-only duration measured before animation playback. Grid topology, browser state, device capability, and heuristic quality can all affect the observed values, so a lower visited count or a shorter time should be interpreted alongside the algorithm's correctness guarantee and returned route cost.

Fair-comparison rule: Keep the start, end, walls, weights, and movement rules constant when comparing algorithms. On a weighted board, compare least-cost guarantees and path cost; on a uniform-cost board, compare minimum-hop behavior. Do not use animation speed as a proxy for algorithmic complexity.

6.3 Deployment and Release Validation Workflow

The repository uses a lockfile-based npm dependency set and exposes practical verification commands for a release candidate: type checking and ESLint can be run together with npm run verify, the production bundle can be checked with npm run build, and production dependency auditing can be run with npm run audit:prod. These commands support a disciplined manual release process; they do not constitute a continuous-integration workflow or a committed regression-test suite.

The deployed application is a standard Next.js application on Vercel. Its core visualizer, maze module, Learn page, and Compare page do not require an environment variable. DATABASE_URL is optional and enables only the Share Grid persistence path; when that feature is enabled, the target PostgreSQL migration must be applied separately. The production configuration also uses restrictive response headers, validates Share Grid payloads, hides X-Powered-By, and keeps server-side secrets out of the client bundle. [8], [14]

- Install the locked dependency set, then run the documented type-checking and linting verification command.
- Build the production bundle and confirm that the main visualizer, Learn page, Compare page, and navigation load correctly.
- Run one unweighted search, one weighted search, and one structured maze to check visible feedback and expected route behavior.
- If Share Grid is enabled, configure DATABASE_URL only on the server, apply the database migration, and validate both create and restore flows.
- Deploy through Vercel, open the public application, and repeat one short end-user demonstration before submission.

6.4 Project Availability

Two public access routes are retained for assessment and reproducibility. The live deployment is the appropriate route for interacting with the visualizer and observing the user interface, whereas the source repository is the appropriate route for examining implementation modules, dependencies, configuration, database schema, and deployment files. These links should remain in the final submitted report so that the examiner can verify the delivered project directly.

- **Live application:** https://pathfinding-visualizer-flax-one.vercel.app/
- **GitHub repository:** https://github.com/z-lovejeet/Pathfinding-Visualizer

Source: Authors; project links verified on 20 July 2026.

7. TEAM CONTRIBUTIONS, LIMITATIONS AND FUTURE SCOPE

7.1 Team Responsibility Allocation

The project was developed by five members with complementary responsibilities. The allocation below has been agreed by the team and makes contribution ownership clear for assessment and the oral presentation. Although work in a collaborative project overlaps during integration, the listed ownership areas identify each member's primary responsibility and the part of the system they can explain directly.

Table 7. Team responsibility allocation.

Member	Registration No.	Primary responsibility
Lovejeet Singh	12521613	Project lead; system architecture; React/Three.js integration; deployment coordination.
Sachit Babbar	12521201	Frontend UI/UX; responsive layouts; controls; navigation; visual polish.
Mohammad Asim	12518421	Pathfinding research and implementation; correctness conditions and algorithm explanation.
Shubham Kumar	12502181	Maze generation; MinHeap and Union-Find data structures; grid logic.
Vineetosh Kumar	12502490	Share/database support; quality checks; report, references, and deployment documentation.

Source: Team-approved responsibility allocation.

7.2 Limitations

- The Compare page uses two 2D DOM mini-grids and sequential computation; it is not a synchronized dual-3D visualization feature.
- Weighted terrain and movable endpoints are available in the main visualizer but not in the current Compare editor.
- Timing uses browser performance.now() and should be treated as an in-context educational metric, not a normalized benchmark.
- The Share Grid feature is anonymous; a UUID is an identifier, not an authorization mechanism.
- Sharing persists only walls, weights, title, and selected algorithm for the default-size board; it does not restore camera, animation, endpoints, or dimensions.
- The main visualizer relies on WebGL; low-power environments would benefit from a lightweight 2D fallback.
- The project currently relies on static verification and manual functional validation rather than a committed regression-test suite or CI workflow.

7.3 Future Scope

- Rebuild Compare with two synchronized React Three Fiber scenes, a shared configuration surface, and an optional time comparison chart while preserving fair same-board input.
- Add weighted terrain, endpoint placement, reproducible random seeds, and scenario presets to Compare so that cost-aware algorithms can be evaluated directly.
- Introduce accessible 2D fallback rendering, improved keyboard guidance, and stronger color-blind-safe alternatives without removing the established visual palette.
- Add optional authentication, ownership, expiry, privacy controls, rate limiting, and monitoring around public shared links.
- Support configurable grid dimensions, diagonal movement modes, terrain presets, and more advanced algorithms such as Jump Point Search or Contraction Hierarchies only when their assumptions are explained clearly.
- Create a formal automated test strategy and continuous integration workflow if project scope and maintenance requirements expand.
- Develop standardized repeatable benchmark scenarios that separate algorithm computation, rendering cost, and animation duration.

7.4 Submission and Demonstration Readiness

Submission readiness includes a final visual inspection of the deployed application, replacement of marked screenshot placeholders with authentic team captures, and confirmation that the public project links in Section 6.4 open correctly. During a viva, the team can use one prepared weighted board to contrast BFS with A*, generate a maze, and connect the observed behavior to the Learn or Compare explanations. The final submission distinguishes source-level validation from manual interaction checks and does not claim automated coverage that the repository does not contain.

Viva recommendation: Let each member explain their primary contribution, then use one short live run to connect the theory: configure a weighted board, compare BFS with A*, generate a maze, and show the Learn or Compare page.

Report note

8. CONCLUSION

Pathfinder demonstrates that data-structure and algorithm concepts become easier to understand when their decisions are made visible. The project models a common four-directional graph, supports semantic board editing, and shows how different strategies explore the same problem. It correctly distinguishes uniform-cost shortest-hop algorithms from non-negative weighted least-cost algorithms: BFS and Bidirectional BFS are appropriate when every move has equal cost; Dijkstra and A* provide cost-optimal routes when weighted terrain is introduced; DFS and Greedy Best-First Search expose useful speed-versus-quality trade-offs without claiming optimality.

The maze module extends this learning environment by generating distinct obstacle topologies. Recursive Division emphasizes chambers and walls; Recursive Backtracker creates deep corridors; Randomized Prim's grows branching frontiers; Randomized Kruskal's applies disjoint-set logic; and Random Scatter intentionally allows no-path examples. Reachability repair ensures that the four structured generators remain usable for the selected endpoints while preserving an honest distinction between endpoint connectivity and the strict mathematical definition of a perfect maze.

From an engineering perspective, the system separates concerns cleanly. Algorithms calculate first, animation explains second, and three instanced mesh layers render the 3D board efficiently. A centralized Zustand store coordinates interaction, result metrics, and safe cancellation. The Learn and Compare pages broaden the educational context, while optional Share Grid persistence is isolated so the main application remains functional without a database. The resulting project is deployable, visually coherent, and ready for further refinement in comparison visualization, accessibility, controlled benchmarking, and secure collaboration features.

The report's final screenshot placeholders should be replaced with the team's own application captures before submission. Once those figures are inserted, the document provides a complete college-level account of the project's motivation, implementation, algorithmic guarantees, validation, contribution allocation, and future scope.

9. REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.

[2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269-271, 1959, doi: 10.1007/BF01386390.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, 1968, doi: 10.1109/TSSC.1968.300136.

[4] R. C. Prim, “Shortest connection networks and some generalizations,” Bell System Technical Journal, vol. 36, no. 6, pp. 1389-1401, 1957, doi: 10.1002/j.1538-7305.1957.tb01515.x.

[5] J. B. Kruskal, Jr., “On the shortest spanning subtree of a graph and the traveling salesman problem,” Proceedings of the American Mathematical Society, vol. 7, no. 1, pp. 48-50, 1956, doi: 10.1090/S0002-9939-1956-0078686-7.

[6] R. E. Tarjan, “Efficiency of a good but not linear set union algorithm,” Journal of the ACM, vol. 22, no. 2, pp. 215-225, 1975, doi: 10.1145/321879.321884.

[7] J. Buck, Mazes for Programmers: Code Your Own Twisty Little Passages. Raleigh, NC: Pragmatic Bookshelf, 2015.

[8] Vercel, “Next.js App Router Documentation.” [Online]. Available: https://nextjs.org/docs/app. Accessed: Jul. 19, 2026.

[9] pmndrs, “React Three Fiber: Introduction.” [Online]. Available: https://r3f.docs.pmnd.rs/getting-started/introduction. Accessed: Jul. 19, 2026.

[10] Three.js Authors, “Three.js Manual: Creating a Scene.” [Online]. Available: https://threejs.org/manual/en/creating-a-scene.html. Accessed: Jul. 19, 2026.

[11] pmndrs, “Zustand Documentation.” [Online]. Available: https://zustand.docs.pmnd.rs/. Accessed: Jul. 19, 2026.

[12] Drizzle Team, “Drizzle ORM with Neon Postgres.” [Online]. Available: https://orm.drizzle.team/docs/connect-neon. Accessed: Jul. 19, 2026.

[13] Neon, “Neon Serverless Driver.” [Online]. Available: https://neon.com/docs/serverless/serverless-driver. Accessed: Jul. 19, 2026.

[14] Vercel, “Next.js on Vercel.” [Online]. Available: https://vercel.com/frameworks/nextjs. Accessed: Jul. 19, 2026.

[15] Recharts Group, “Recharts Documentation.” [Online]. Available: https://recharts.github.io/en-US/guide/. Accessed: Jul. 19, 2026.

[16] Shiki Authors, “Shiki Installation and Usage.” [Online]. Available: https://shiki.style/guide/install. Accessed: Jul. 19, 2026.

APPENDIX A. SCREENSHOT INSERTION CHECKLIST

This appendix gives the final submission workflow for the marked screenshot areas. The document contains embedded placeholders rather than borrowed web images so the submitted report can contain authentic evidence from the team's own deployed project. In Microsoft Word, click the placeholder image, press Delete, then use Insert > Pictures to place the matching screenshot. Keep the caption immediately below the image and update the content if the captured view differs materially from the suggested description.

Appendix Table A1. Screenshot replacement guide.

Figure	What to capture	Checklist before export
Figure 4	Full main visualizer interface.	Show control panel, clear 3D board, legend, and result panel; avoid clipped UI.
Figure 5	Completed search result.	Ensure violet explored nodes and amber final path are visible; include metrics if readable.
Figure 6	Maze generation result.	Prefer a labeled four-panel collage so each structured maze has a visible example.
Figure 7	Learn and Compare evidence.	Show Shiki pseudocode/accordion and the current 2D comparison grids/chart.
Figure 8	Optional sharing/deployment.	Use only if Share Grid is configured or demonstrate the public Vercel deployment.

Source: Authors.

Suggested Screenshot Naming

- fig4-main-visualizer.png
- fig5-completed-path.png
- fig6-maze-generation-collage.png
- fig7-learn-compare.png
- fig8-share-or-deployment.png (optional)

Final Submission Checks

- Replace or remove every screenshot placeholder before final submission.
- Ensure all figures remain readable after Word exports the report to PDF.
- Update the Contents page numbers in Word if screenshot insertion changes pagination.
- Keep the source lines under figures honest: use "Source: Authors" for your own captures.
- Confirm that the live and GitHub links in Section 6.4 open correctly before final submission.